

Experiment No.	2
Experiment Title	K-Nearest Neighbors and Naïve Bayes Classification ¹
Student Name	Naren Kandasamy
Register Number	3122247001036
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

The purpose of this experiment was to compare the performance of different variations of the Naïve Bayes algorithm (Gaussian, Multinomial, and Bernoulli) against a standard K-Nearest Neighbors (KNN) model. Utilizing the Kaggle Spambase dataset, the experiment also involved tuning the hyperparameter k , assessing the computational differences between KDTree and BallTree structures, and employing Cross Validation to ensure the reliability of the results.

2 Problem Statement

The primary objective of this task was to classify emails as either "Spam" or "Ham" based on the frequency of specific words and characters appearing in the text. The input dataset consisted of a table containing 57 numerical features (representing word frequencies), while the expected output was a binary prediction where 1 indicates spam and 0 indicates a normal email.

3 Dataset Description

Dataset Name	Kaggle Spambase Dataset (<code>spambase.csv.csv</code>)
Dataset Source	Kaggle / UCI Machine Learning Repository
Number of Samples	4,601
Number of Features	57 (Word/Character Frequencies and lengths)
Number of Classes	2 (Spam, Ham)
Missing Values	0
Train-Test Split	Train: 3,220 — Validation: 690 — Test: 691

4 Exploratory Data Analysis

An initial examination of the dataset was conducted to understand its structure. The vocabulary size comprised 56 predictive features alongside a single target label. A class distribution chart was plotted to confirm the balance between spam and ham emails, and it was determined that the dataset was reasonably balanced, allowing the analysis to proceed without extensive resampling.

5 Data Preprocessing

Preparing the data for modeling required a few specific processing steps:

- **Handling Sparsity:** Given that the data consisted of text frequencies, the standard mean-imputation and zero-mean scaling were omitted for the Naïve Bayes models. This decision was made to preserve the exact frequency counts of the words without introducing artificial distortion.
- **Scaling for KNN:** Because KNN relies on calculating physical distances between points, it is highly sensitive to unscaled data. A `MinMaxScaler` was applied to both the training and testing sets prior to fitting the KNN model to ensure fair distance weighting.
- **Train-test split:** The dataset was divided into Training, Validation, and Testing sets using a stratified approach, ensuring that the ratio of spam to ham remained consistent across all splits.

6 Machine Learning Algorithm

Two main types of algorithms were evaluated for this classification task:

1. Naïve Bayes (Gaussian, Multinomial, Bernoulli):

- **Working principle:** A probabilistic classifier based on Bayes' theorem. It operates on the strict assumption that every feature is completely independent of the others, which, despite being an oversimplification for text data, often yields strong results in practice.
- **Mathematical formulation:** $P(c|x) = \frac{P(x|c)P(c)}{P(x)}$ where c represents the class and x represents the feature vector.

2. K-Nearest Neighbors (KNN):

- **Working principle:** Rather than constructing an underlying probability model, this instance-based algorithm examines the k closest data points in the training set and assigns a class based on a majority vote.
- **Mathematical formulation:** Distance is typically calculated using the Euclidean formula: $d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$.
- **Tree Structures:** KDTree and BallTree structures were utilized to accelerate the distance computations, as calculating distances to every single point iteratively becomes computationally expensive.

7 Experimental Setup

Python Version	3.10+
Scikit-Learn Version	1.0+
IDE	Jupyter Notebook / VS Code
Operating System	Linux
Hardware	Standard CPU Workspace

8 Hyperparameter Selection

KNN Comparison (Varying k): Several different values for k (1, 3, 5, 7, 9, 11) were tested. Prior to the introduction of distance weighting, the highest accuracy observed was at $k = 5$ (approximately 90.3%).

Optimization Strategy	Result
GridSearchCV Best Params	<code>{'n_neighbors': 7, 'weights': 'distance'}</code> (Found)
RandomizedSearchCV Best Params	<code>{'n_neighbors': 5, 'weights': 'distance'}</code> (Found)

9 Results

1. Model Performance Comparison:

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Multinomial NB	0.8958	0.9464	0.7794	0.8548	0.9558
Gaussian NB	0.8220	0.7018	0.9522	0.8081	0.9166
Bernoulli NB	0.6483	0.8717	0.1250	0.2186	0.6097
Optimized KNN	0.9088	0.9023	0.8654	0.8835	0.9412

2. 5-Fold Cross Validation (Best NB vs Best KNN):

Fold	Naïve Bayes	Best KNN
1	0.8825	0.9055
2	0.8964	0.9130
3	0.8849	0.9015
4	0.8900	0.9105
5	0.9015	0.9118
Average	0.8911	0.9085

3. Tree Structure Timing Comparison:

Algorithm	Train Time (s)	Predict Time (s)
KDTree	0.0107	0.1106
BallTree	0.0071	0.0942

10 Result Visualization

A series of plots were generated to visualize model performance during execution:

- **Accuracy vs k :** A line chart illustrating the fluctuation in accuracy as the value of k varied from 1 to 11.
- **ROC Curves & PR Curves:** These curves facilitated the comparison of the area under the curve across the different Naïve Bayes models and the optimized KNN model.

- **Confusion Matrices:** Grids were generated to identify where the models were producing false positives versus false negatives.
- **GridSearchCV Heatmap:** A heatmap visualization clearly displaying which combinations of k and weight parameters yielded the optimal F1-Score.

11 Discussion

The results revealed that Multinomial Naïve Bayes was highly effective, achieving nearly 90% test accuracy and an ROC-AUC of 0.9558. However, after tuning the KNN classifier using GridSearchCV (which identified optimal settings of $k = 7$ and distance-based weights), the KNN model outperformed Naïve Bayes. It achieved a stable average accuracy of 90.85% across the 5-fold cross-validation. Additionally, a comparison of the two tree algorithms showed that BallTree was marginally faster than KDTree for making predictions on this specific dataset.

12 Conclusion

The experiment demonstrated that while probabilistic models such as Naïve Bayes are highly computationally efficient and perform well on text data, distance-based algorithms like KNN can yield superior classification performance when the data is appropriately scaled and hyperparameters are carefully tuned. Both models ultimately proved successful at distinguishing between spam and ham emails.

13 References

1. Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 2011.